

DREAMAVOID: Critical-Phase Test-Time Dreaming to Avoid Failures in VLA Policies

Xianzhe Fan¹ Yuxiang Lu¹ Shenyuan Gao² Xiaoyang Wu¹
 Ruihua Han¹ Manling Li³ Hengshuang Zhao¹
¹HKU ²HKUST ³Northwestern University

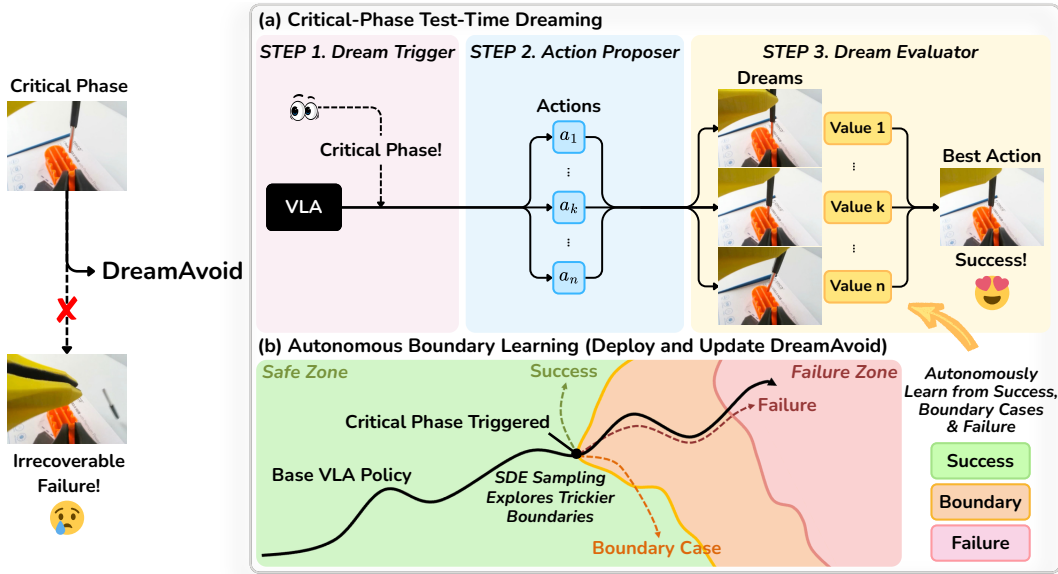


Figure 1: We propose DREAMAVOID, (a) a *critical-phase test-time dreaming* framework that enables the VLA to anticipate and avoid failure. This framework consists of a Dream Trigger, an Action Proposer, and a Dream Evaluator. (b) We also introduce an *autonomous boundary learning* paradigm to equip the world model-based Dream Evaluator with failure awareness.

Abstract

Vision-Language-Action (VLA) models are often brittle in fine-grained manipulation, where minor action errors during the critical phases can rapidly escalate into irrecoverable failures. Since existing VLA models rely predominantly on successful demonstrations for training, they lack an explicit awareness of failure during these critical phases. To address this, we propose DREAMAVOID, a critical-phase test-time dreaming framework that enables VLA models to anticipate and avoid failures. We also introduce an autonomous boundary learning paradigm to refine the system’s understanding of the subtle boundary between success and failure. Specifically, we (1) utilize a Dream Trigger to determine whether the execution has entered a critical phase, (2) sample multiple candidate action chunks from the VLA via an Action Proposer, and (3) employ a Dream Evaluator, jointly trained on mixed data (success, failure, and boundary cases), to “dream” the short-horizon futures corresponding to the candidate actions, evaluate their values, and select the optimal action. We conduct extensive evaluations on real-world manipulation tasks and simulation benchmarks. The results demonstrate that DREAMAVOID can effectively avoid failures, thereby improving the overall task success rate. Our code is available at <https://github.com/XianzheFan/DreamAvoid>.

1 Introduction

Vision-Language-Action (VLA) models provide a promising framework for general-purpose robotic manipulation by directly mapping multimodal observations and language instructions into the action space [1–3]. However, they remain fragile in fine-grained manipulation scenarios. In tasks such as insertion, precise alignment, and contact-sensitive placement, a small local action error can rapidly escalate into an irrecoverable failure [4–6]. In many cases, the policy does not perform poorly over the entire trajectory; rather, failures are often triggered by brief yet low-tolerance *critical phases* [7]. How can we equip VLAs with stronger consequence-awareness during these critical phases? In human cognition, complex physical tasks are often balanced between fast, intuitive execution and slow, deliberate foresight. When threading a needle or plugging in a cord, we do not consciously plan every minute movement; we rely on intuitive motor priors. However, at critical moments, we briefly pause and simulate in our minds, “dreaming” of the physical consequences our actions are about to bring, to avoid making mistakes.

Inspired by this, introducing additional computation during the test time to endow VLAs with a “dreaming” capability emerges as a natural choice. Powerful large-scale pre-trained world models [8] can accurately capture complex environmental dynamics and generate high-fidelity future simulations based on candidate actions. Can we leverage this capability to achieve better consequence-awareness for robots through dreaming at test-time? We conducted a pilot study on a charger plugging task, comparing two test-time compute paradigms: (1) Direct Semantic Evaluation, which uses a VLM to predict success or failure by receiving past and current visual observations and future numerical action sequences; and (2) Explicit Spatiotemporal Dreaming, which uses a large-scale pre-trained action-conditioned world model to explicitly forward-render actions into future visual frames, which are then evaluated by the same VLM. The results showed that when faced with abstract numerical inputs, the VLM tends towards a conservative “almost certain to fail” prediction pattern, whereas its accuracy improves significantly when evaluating the visual simulations provided by the world model. This contrast reveals a key insight: for fine-grained manipulation, relying solely on implicit numerical reasoning cannot bridge the gap between abstract actions and physical consequences. Therefore, rendering actions into high-fidelity future frames for explicit test-time dreaming is highly beneficial.

Following the idea of introducing extra computation at test-time, several studies have proposed their own solutions. For instance, RoboMonkey utilizes a VLM as an online verifier to score and filter candidate actions [9]. Some works leverage action-conditioned models for always-on online search and reranking [10–12]. On the other hand, some research achieves dynamic scaling of test-time compute through implicit iterative reasoning within a continuous latent space [13]. However, implementing test-time computation in robotic manipulation tasks still faces two major challenges. First, always-on inference is prohibitively expensive [14]; executing heavy rollouts at every time step introduces severe latency [12, 13]. Second, existing VLAs typically rely on successful trajectories for training [1]. This results in policies that learn to imitate success but lack explicit supervision regarding failure boundaries [3, 15]. This deficiency is particularly prominent during critical phases, where the robot needs to distinguish between subtle action differences that carry significant consequences.

To address these challenges, we propose DREAMAVOID, a critical-phase test-time dreaming framework designed to enhance the ability of VLAs to avoid failures (Figure 1). We also introduce an autonomous boundary learning paradigm. Specifically, (1) DREAMAVOID proposes a selective intervention mechanism. During routine execution, the robot follows the base policy. When a lightweight Dream Trigger detects an impending critical phase, the system invokes an Action Proposer to sample multiple diverse candidate action chunks. Subsequently, a world-model-based Dream Evaluator engages in short-horizon dreaming to predict their future visual evolution and values, selecting the optimal action. (2) DREAMAVOID proposes an autonomous boundary learning paradigm, introducing failure and boundary data from which the world model can benefit. For the dreams to truly serve a preventive purpose, the world model must possess the ability to have “nightmares.” If trained solely on success data, the world model would generate optimistic hallucinations due to a lack of negative boundary constraints. By deploying DREAMAVOID to explore and collect success, failure, and boundary data, we teach the world model the boundary between success and failure.

We evaluate DREAMAVOID across multiple real-world tasks that require precise alignment, stable contact, and effective failure avoidance. Additionally, we conduct simulation benchmarking on LIBERO [16] and SimplerEnv [17]. The experimental results demonstrate that DREAMAVOID consistently improves task success rates compared to base VLA policies. These findings indicate that

test-time compute does not need to be always-on; as long as it is invoked during critical phases, world model-based test-time dreaming can implement corrections before failures actually occur, thereby enhancing the reliability of VLAs.

The contributions of our paper are summarized as follows: (1) We emphasize the focus on critical phases and propose DREAMAVOID, a critical-phase test-time dreaming framework that enables VLAs to foresee and avoid failures in advance. This plug-and-play framework consists of a Dream Trigger, an Action Proposer, and a Dream Evaluator. (2) We propose an autonomous boundary learning paradigm that endows the world model with failure awareness by introducing failure and boundary data. (3) We demonstrate the advantages of explicit spatiotemporal dreaming in failure prediction through a pilot study. We conduct extensive evaluations on real-world manipulation tasks and simulation benchmarks. The results show that DREAMAVOID can effectively intervene before minor execution errors deteriorate into irrecoverable failures, improving overall task success rates.

2 Related Work

Brittle VLA Fine-Grained Manipulation. VLA models combine large-scale vision-language pretraining with robotic action generation, substantially improving robots’ semantic understanding and cross-task generalization abilities [1–3]. However, recent studies suggest that the bottleneck of VLA models often lies not in insufficient global semantic understanding, but in local decision errors at a few highly sensitive moments [4]. As a result, VLA models remain brittle in tasks that require fine-grained alignment, contact control, and obstacle avoidance [5, 6]. Building on this observation, this paper treats failures in fine-grained manipulation as a temporally localized critical phase problem.

Test-Time Compute. To enhance deployment robustness, existing works have begun introducing additional computation into the testing phase. The differences between our method and existing test-time compute mainly lie in two aspects. (1) In terms of trigger timing, existing works tend to favor always-on computation [10, 13, 18–20]. TT-VLA [18] and EVOLVE-VLA [19] utilize dense progress rewards for constant online policy adaptation during inference. RD-VLA achieves adaptive computation depth through latent recurrent inference [13]. In contrast, our method DREAMAVOID directly executes the base VLA policy most of the time, triggering additional computation only when the Dream Trigger predicts entry into a temporally sparse yet consequence-sensitive critical phase. (2) In terms of evaluation methods, most Verifier or MPC works predominantly rely on current-state scoring or pixel-level goal alignment [9, 11]. Visual MPC compares candidate futures based on action-conditioned video prediction, but its scoring criterion is the probability of designated pixels reaching predefined target positions [11]. RoboMonkey investigates VLA test-time scaling by Gaussian perturbation sampling and selecting from candidate actions using a VLM-based verifier [9]. GPC combines Diffusion Policy [21] with a world model to evaluate candidate actions during deployment, but it does not leverage the rich physical priors embedded in large-scale pre-trained world models [10]. In comparison, we utilize the high-fidelity future simulation capability of large-scale pre-trained world models. Instead of scoring candidate actions solely based on the current state or generic verifier signals, we explicitly compare the future consequences of different action chunks through failure-aware short-horizon dreaming before performing ranking and selection.

3 Pilot Study: Do We Need Dreaming to Avoid Failure in Critical Phases?

Table 1: Pilot study on failure prediction for fine-grained manipulation (Charger Plugging).

Paradigm	Action Representation	Evaluator	Accuracy (%)	Precision (fail) (%)	Recall (fail) (%)
Direct Semantic Evaluation	Textified Action Sequence	Gemini 3.1 Pro	57.5	56.4	100.0
Explicit Spatiotemporal Dreaming	DreamDojo-Generated Video	Gemini 3.1 Pro	80.0	88.9	72.7

We first explore the paradigm of predicting fine-grained manipulation failures at test time. Given the strong semantic reasoning capabilities of VLMs, an intuitive baseline is to directly use them as consequence predictors during test time. We conducted a pilot study on the Charger Plugging task, which is highly sensitive to minor deviations. We constructed a test set containing 40 data episodes (18 successes and 22 failures), which were collected by rolling out a base policy $\pi_{0.5}$ [2] trained on 100 human teleoperation demonstrations in the real world. To accurately evaluate

the model’s reasoning ability during the critical phase, human annotators manually located a key interaction frame t_{crit} within each test trajectory. This moment is defined as a frame within the 1-2 s immediately preceding the end-effector’s decisive physical contact with the target object (e.g., just before the plug aligns with the socket). At each t_{crit} , we extract the current frame o_t and a historical observation sequence. The action input is the downsampled future action chunk u_t . We compare two test-time failure prediction paradigms: (1) Direct Semantic Evaluation: We input the visual observation sequence containing the historical and current frames, along with the textified action sequence and its semantic description, into Gemini 3.1 Pro [22]. We use few-shot prompting [23] for it to predict end-to-end whether executing u_t will result in a task failure (such as a collision or misalignment). (2) Explicit Spatiotemporal Dreaming: We use DreamDojo [8], an action-conditioned, large-scale pre-trained world model, to forward-render u_t into future video frames, which are then evaluated by Gemini 3.1 Pro acting as a visual evaluator. Details are provided in Appendix B.

The experimental results reveal the differences in prediction reliability between the two paradigms (Table 1). The accuracy of the Direct Semantic Evaluation paradigm is only 57.5%. Analysis shows that when faced with abstract numerical inputs, the VLM tends toward a conservative “almost always predict failure” pattern: it successfully identified all 22 failure samples, but also misclassified 17 successful samples as failures. Although the VLM can understand the semantic goal of “Plug the charger into the socket,” it is almost entirely unable to establish an effective connection between the raw numerical action chunk and its actual physical consequences. In fine-grained manipulation, even a tiny deviation in the action trajectory can lead to catastrophic collisions. However, these minuscule numerical differences are indistinguishable to the VLM, rendering it unable to accurately predict the physical evolution of the trajectory. Conversely, Explicit Spatiotemporal Dreaming demonstrated stronger predictive capabilities, achieving an accuracy of 80.0%. Although slightly optimistic (missing 6 failures), it drastically suppressed false alarms (misclassified success cases dropped from 17 to 2). By explicitly simulating numerical actions forward into visual futures, the world model serves as a critical spatiotemporal bridge. It translates abstract numerical errors into explicit visual deviations, for example, directly dreaming of the plug hitting the edge of the socket or the tabletop. Subsequently, Gemini 3.1 Pro can easily detect this visual anomaly from the generated high-fidelity frames. This pilot study demonstrates that explicit spatiotemporal dreaming is important for avoiding failure in critical phases.

4 DREAMAVOID

Based on the insights from the pilot study (§3), we propose DREAMAVOID, a critical-phase test-time dreaming framework for VLA manipulation. The core idea is that the robot directly executes the base VLA policy during most time steps, and only triggers test-time dreaming when it predicts an imminent transition into a critical phase. Specifically, DREAMAVOID consists of three modules: (1) **Dream Trigger**, which monitors the current state, identifies critical phases, and issues intervention commands; (2) **Action Proposer**, responsible for sampling multiple action chunks from VLA; (3) **Dream Evaluator**, which utilizes a world model to forward-simulate the short-horizon future of these candidate actions, and integrates a value model to score and rank them. Through this design, DREAMAVOID allocates compute-intensive failure-aware reasoning to the truly important moments.

Problem Formulation. We consider a trajectory $\tau = \{(o_t, l, a_t)\}_{t=1}^T$, where o_t denotes the visual observation at time t , l is the language instruction, and a_t is the robot action. Parameterized via **flow matching**, the base VLA policy π_θ predicts an action chunk of horizon H , $u_t = [a_t, a_{t+1}, \dots, a_{t+H-1}]$, such that $u_t \sim \pi_\theta(\cdot \mid o_t, l)$. Although directly executing the base policy is computationally efficient, it is susceptible to local errors during critical phases. Conversely, applying test-time compute or forward simulation at every time step incurs immense computational overhead and latency. Therefore, our problem transforms into designing an adaptive inference strategy: the system identifies critical phases within τ to selectively allocate test-time compute.

4.1 Dream Trigger

The first step of DREAMAVOID is to determine whether the current state has entered a critical phase. We introduce a lightweight Dream Trigger $p_t = f_\phi(\mathcal{O}_{t-K+1:t}, s_t)$, whose inputs are (1) single- or multi-view observations from the recent K frames $\mathcal{O}_{t-K+1:t} = \{o_{t-K+1}, \dots, o_t\}$, and (2) current proprioceptive state s_t . The output of the Dream Trigger is the probability $p_t \in [0, 1]$ that the current

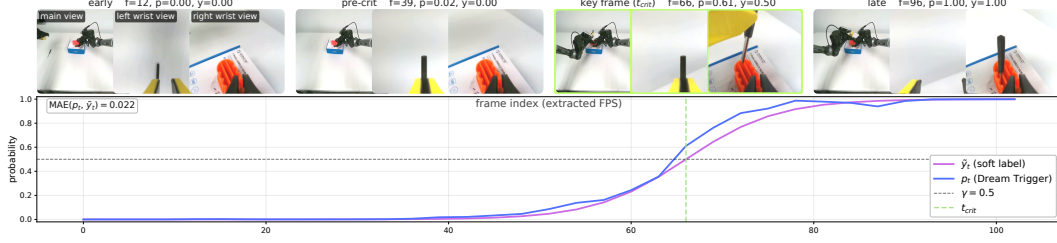


Figure 2: Qualitative example of the Dream Trigger ($\beta = 5$ frames in the soft label $\tilde{y}_t$).

state has entered a critical phase. After collecting data via teleoperation, we manually annotate the timestamp t_{crit} of entering the critical phase. Directly using a hard label $y_t = \mathbb{I}[t \geq t_{crit}]$ is susceptible to human annotation bias, so we design a soft label $\tilde{y}_t = \text{Sigmoid}((t - t_{crit})/\beta)$. This is equivalent to applying a soft transition within the range of $[t_{crit} - \beta, t_{crit} + \beta]$. The Dream Trigger is supervised using binary cross-entropy with class-balanced weights:

$$\mathcal{L}_{trigger} = -\frac{1}{T} \sum_{t=1}^T [w_+ \tilde{y}_t \log p_t + (1 - \tilde{y}_t) \log(1 - p_t)]. \quad (1)$$

The weight $w_+ = N_-/N_+$ is used to compensate for the critical / non-critical frame ratio. For each real-world or simulation task in this paper, we select 100 videos for t_{crit} annotation. The triggering condition for the Dream Trigger is $c_t^{\text{test}} = \mathbb{I}[p_t \geq \gamma]$, where γ is the trigger threshold. When $c_t^{\text{test}} = 0$, the system is in the regular phase and directly executes the action chunk output by the base policy, ensuring execution efficiency and coherence. Conversely, when $c_t^{\text{test}} = 1$, the Dream Trigger instructs the system to enter the critical phase (Figure 2). See Appendix C.1 for details.

4.2 Action Proposer

When the system determines that it is currently in a critical phase, DREAMAVOID does not discard the base VLA policy, but instead continues to use it as a generator for candidate actions. In this paper, the base policy is modeled using flow matching, progressively mapping noise into an action chunk by integrating a conditional velocity field within the action space. During standard inference, this generation process corresponds to a deterministic probability flow ODE trajectory; thus, given an observation and language instruction, it typically outputs only one default action chunk. To obtain multiple candidate action chunks during testing, DREAMAVOID replaces this deterministic ODE inference process with its corresponding stochastic SDE sampling process [24, 25]. Standard flow matching inference corresponds to the following probability flow ODE:

$$du = v_\theta(u, \rho \mid o_t, l) d\rho, \quad (2)$$

where $\rho \in [0, 1]$ denotes the internal flow matching time variable, and v_θ is the conditional velocity field that determines the deterministic evolution trajectory of the action chunk from the initial noise distribution to the target distribution. To support multi-candidate sampling during testing, we reformulate it into the corresponding stochastic process:

$$du = v_\theta(u, \rho \mid o_t, l) d\rho + \sigma(\rho) dw_\rho, \quad (3)$$

where $\sigma(\rho)$ controls the sampling noise intensity, and w_ρ denotes the Wiener process. In practice, we adopt a constant noise scheduling strategy. The specific values of σ for different tasks can be found in Table 3, Appendix C.2. By repeating this stochastic sampling process, we can generate multiple candidate action chunks for the same time step to be used for the Dream Evaluator. This design serves two primary purposes. First, all candidate actions still originate from the velocity field of the same pre-trained policy; therefore, they are not blind searches in an unconstrained action space, but are consistently bounded by the task semantics and action priors encoded in the base policy. Second, the randomness introduced by SDE allows the system to expose multiple locally plausible action hypotheses, which would often collapse into a single prediction under standard ODE inference. This is particularly crucial during critical phases: multiple seemingly reasonable action chunks might lead to drastically different downstream outcomes. For example, one candidate might accomplish precise alignment, while another might cause lateral deviation, collision, or object slipping.

4.3 Dream Evaluator

Action-conditioned world models provide a natural mechanism for evaluating the subsequent effects of candidate actions. DreamDojo [8], which is pre-trained on large-scale videos and post-trained on robotics data, provides rich physical priors of manipulation tasks for test-time dreaming. Specifically, for each candidate action chunk $u_t^{(k)}$ and current observation o_t , we use the distilled DreamDojo (Appendix C.3.1) to predict its corresponding future observation sequence $\hat{y}_t^{(k)} = \mathcal{W}_\psi(o_t, u_t^{(k)})$. We define the value of a candidate action chunk as the real progress delta it brings within the short-horizon window. Built upon a VLM trained on a large-scale dataset, Robometer [26] acts as a general-purpose robotic reward model capable of predicting frame-level task progress $\eta \in [0, 1]$. This allows us to automatically compute the continuous labels $z_t^{(k)}$ for the corresponding trajectories (Appendix C.3.2). During the training of the value model, the model needs to learn accurate physical consequences from clear, real-world videos. However, during test time, it only receives video prediction $\hat{y}_t^{(k)}$ generated by DreamDojo, which may contain blurriness or artifacts. To balance learning efficiency and overcome this domain shift, we employ a *real-dream joint training* strategy. When constructing the inputs for the task-specific value model, (1) we directly input the *real future sequence* y_t^{true} into the model (fitting $V_\omega(o_t, u_t, y_t^{\text{true}}) \rightarrow z_t$), enabling the network to learn precise value mappings based on lossless visual features; (2) we input (o_t, u_t) into DreamDojo to generate *dreams* $\hat{y}_t$ and use them as inputs as well (fitting $V_\omega(o_t, u_t, \hat{y}_t) \rightarrow z_t$). By treating the generated sequences as a form of in-domain data augmentation, we force the value model to learn to extract crucial physical progress features despite the visual imperfections of DreamDojo. During this process, the ground-truth labels z_t required for the aforementioned fitting are all obtained by using Robometer to score the real future trajectories y_t^{true} sampled from the offline dataset.

Furthermore, the vast majority of routine offline teleoperation trajectories are in a state of steady progression (exhibiting small positive progress deltas). Direct random sampling would cause the model to lose its discriminative power for dangerous situations. To address this, we employ priority-based oversampling when constructing the training batch $\mathcal{B}$: 20% of the data is sampled from the terminal state horizons of successful completions, 40% is drawn from the additionally collected boundary and failure data (see §4.4, including crash data penalized to $z = -1$), and the remaining 40% comes from routine steady progression phases. We jointly use a Huber robust regression loss [27] and a continuous margin ranking loss:

$$\mathcal{L}_{\text{VE}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} L_\delta(\hat{v}^{(i)}, z^{(i)}) + \frac{\lambda_{\text{rank}}}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \max\left(0, -(\hat{v}^{(i)} - \hat{v}^{(j)}) + \alpha(z^{(i)} - z^{(j)})\right), \quad (4)$$

where $\hat{v}^{(i)}$ and $z^{(i)}$ are the predicted progress delta and the ground truth for the i -th flattened sample in batch $\mathcal{B}$, respectively. $\mathcal{P} = \{(i, j) \in \mathcal{B} \mid z^{(i)} > z^{(j)}\}$ is the set of valid training pairs, L_δ denotes the Huber loss, λ_{rank} balances the two loss terms, and α is a scaling factor requiring the network’s predicted score difference to open up a corresponding margin. During testing, the Dream Evaluator selects the candidate action with the highest predicted progress delta $\hat{v}^{(k)}$.

4.4 Dream the Boundaries: Autonomous Learning from Success, Failure & Boundary Cases

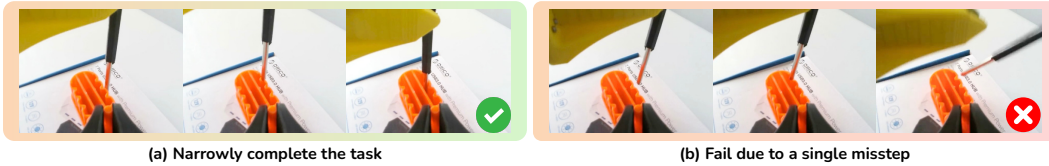


Figure 3: Boundary data in $\mathcal{D}_{\text{online}}$.

DREAMAVOID introduces failure and boundary data to generate discriminative and informative “dreams.” Let $\mathcal{D}^+$ and $\mathcal{D}^-$ denote the sets of successful and failed trajectories, respectively. The base policy is trained exclusively on $\mathcal{D}^+$, ensuring that its intuitive actions while “awake” only imitate effective behaviors, thereby preventing failure modes from being mixed into the action distribution. Conversely, the Dream Evaluator, the engine for test-time dreaming, is jointly trained on $\mathcal{D}^+ \cup \mathcal{D}^-$. The motivation behind this design is that, if trained only on successful data, the world model will

tend to generate “optimistic dreams” (as observed in §3 and related work [28, 29]). It easily learns what success looks like, but struggles to clearly identify which locally feasible actions will actually lead to failure. Especially in fine-grained manipulation tasks, visually subtle deviations can result in drastically different outcomes. By incorporating failure trajectories, the world model effectively learns the critical boundaries between success and failure.

However, human teleoperation datasets $\mathcal{D}_{teleop}$ primarily consist of successful demonstrations, making it difficult to provide informative failure cases. To endow the world model with true failure awareness, we propose an *autonomous boundary learning* paradigm. We deploy the initial version of DREAMAVOID (trained only on $\mathcal{D}_{teleop}$), obtaining a mixed dataset $\mathcal{D}_{online}$ that contains successful, boundary, and failed trajectories. At critical phases, the Action Proposer actively explores the action manifold and generates diverse candidate actions. Due to the optimistic hallucinations of the initial Dream Evaluator, it inevitably misjudges locally plausible but globally dangerous actions as safe, thereby naturally exposing the system to trickier boundary cases. These boundary data can either result in narrowly completing the task or lead to failure by a single misstep (Figure 3). The collected empirical data is assigned value labels z_t (defined in §4.3). Subsequently, we combine $\mathcal{D}_{online}$ with $\mathcal{D}_{teleop}$ to fine-tune the Dream Evaluator.

5 Real-World Experiments

We conduct real-world experiments on the AgileX PiperX robotic platform, which is equipped with one front camera (RealSense D455) and two wrist cameras (RealSense D435), as shown in Figure 4. The robot receives RGB observations and language instructions, and executes the action chunks output by the policy. We design four real-world manipulation tasks for evaluation that require accurate spatial alignment, stable contact establishment, and failure avoidance, including **Cup Sleaving**, **Charger Plugging**, **Cap Opening**, and **Screw Insertion**. Compared to typical scenarios, these tasks are much more sensitive to small action deviations. The specific task instructions and sample observations from the three viewpoints can be found in Table 4 and Appendix D.2.

Evaluated Methods. To conduct a comprehensive evaluation, we compare DREAMAVOID with baseline methods and introduce different variants of DREAMAVOID. We compare DREAMAVOID with the following baselines: (1) Base VLA policy: directly executes the action chunk output by $\pi_{0.5}$. (2) GPC-RANK [10]: this method constantly samples multiple candidate action chunks during testing and utilizes a world model for online ranking. We use GPC-RANK as the primary baseline to validate the effectiveness of several designs in DREAMAVOID: First, regarding the trigger timing, GPC-RANK is a computationally intensive always-on intervention, whereas DREAMAVOID implements a more efficient critical-phase intervention. Second, in terms of consequence prediction, GPC-RANK’s world model relies solely on expert data and random exploration for training, while our Dream Evaluator not only possesses physical priors from large-scale pre-training, but also masters an explicit boundary awareness of critical failures through joint training on real failure and boundary data. Finally, unlike GPC-RANK, which relies on explicit state distances or zero-shot VLM scoring, DREAMAVOID adopts more robust progress value evaluation. To ensure experimental fairness, both are evaluated based on the same base policy $\pi_{0.5}$. To demonstrate the contributions of our training paradigm (see §4.3 and §4.4), we define two variants of our method based on the training data distribution: (1) DREAMAVOID-Vanilla (hereafter **DA-Vanilla**): Both the base VLA policy and the Dream Evaluator are trained exclusively on the human teleoperation dataset $\mathcal{D}_{teleop}$. (2) DREAMAVOID-ABL (hereafter **DA-ABL**): This represents the full DREAMAVOID framework incorporating Autonomous Boundary Learning. The base policy remains trained solely on $\mathcal{D}_{teleop}$, while the Dream Evaluator is fine-tuned on the mixed dataset $\mathcal{D}_{teleop} \cup \mathcal{D}_{online}$.

Results. Figure 4 summarizes the real-world results. DREAMAVOID consistently outperforms the base policy and the GPC-RANK method across all tasks. DA-ABL achieves the highest average success rate of 72.5%, surpassing the base policy ($\pi_{0.5}$, 48.8%) by 23.7%. Additionally, it surpasses the baseline method GPC-RANK (54.4%) in average success rate by 18.1%. This demonstrates that selective, failure-aware test-time dreaming with rich physical priors is beneficial. Different variants of DREAMAVOID also reveal the importance of our training paradigm. DA-Vanilla already yields an average improvement of 18.1% over the base policy. However, incorporating autonomous boundary learning (DA-ABL) brings an additional 5.6% performance improvement. Since DA-Vanilla is trained exclusively on successful demonstration trajectories, its world model tends to generate optimistic

dreams, which limits its ability to intervene effectively during critical phases. In contrast, DA-ABL enhances the world model’s failure awareness by incorporating boundary and failure trajectories.

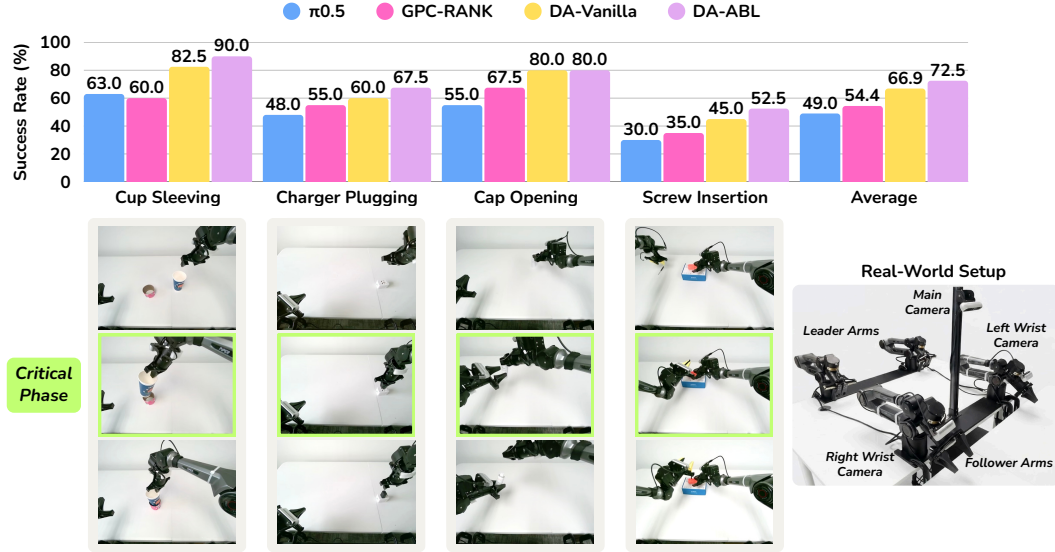


Figure 4: Comparisons in the real world. Each real-world task is evaluated over 40 independent trials.

DREAMAVOID significantly reduces failures caused by lateral drift, premature collisions, and contact misalignment. In several experimental trajectories, the base policy produces reasonable actions at most time steps, but makes locally plausible yet globally detrimental decisions as it approaches the critical phase. By sampling multiple candidate action chunks and explicitly evaluating their short-horizon downstream consequences, DREAMAVOID is able to reject these failure-prone proposals and instead execute actions that are more consistently aligned with task progression.

A key design of DREAMAVOID is to intervene only during the critical phase. Experimental results demonstrate that this strategy achieves a better balance between performance and computational overhead (Table 5, Appendix D.3). Methods like GPC-RANK, which perform candidate evaluation at every timestep, not only incur higher overhead but also cause significant jitter in the robot arm’s trajectory. In contrast, DREAMAVOID achieves higher computational efficiency while maintaining smooth and reliable physical interaction.

Ablation Study. We conduct ablation studies to analyze the contribution of the modules in DREAMAVOID to the performance improvement. All ablation experiments are conducted on real-world manipulation tasks. **(1) Action Proposer vs. Repeated ODE.** We analyze the impact of the candidate action generation method on system performance. To ensure a fair comparison, we construct a strong baseline (Repeated ODE + DA-ABL Dream Evaluator): it retains the Dream Evaluator of DA-ABL, but replaces the candidate action generation mechanism by inputting different random initial noises into the base policy and running deterministic flow inference multiple times to generate the same number of candidate actions. However, Table 2 shows that even when equipped with the strongest Dream Evaluator, the average success rate of Repeated ODE is only 57.5%, which is far inferior to the full method DA-ABL using SDE sampling (72.5%). This aligns with the principle of “garbage in, garbage out”: if the Action Proposer cannot provide high-quality candidate solutions, leading to a severely homogenized candidate pool, then even if the Dream Evaluator can identify impending disasters, it cannot save the system. This indicates that, compared to deterministic trajectories that only alter the noise at the initial timestep (ODE), the continuous random injection of SDE during the denoising process can explore the action manifold more effectively, exposing richer and higher-quality feasible solutions. Such sample diversity is crucial for avoiding subtle failure boundaries during critical phases. **(2) Dream Evaluator vs. Random Select.** We verify the necessity of world-model-based consequence evaluation by replacing the Dream Evaluator with random selection within the SDE candidate pool. Table 2 shows that the average success rate of the *random select* variant is only 45.0%, which is not only much lower than DA-ABL (72.5%), but even lower than the base policy (48.8%). This demonstrates that the candidate diversity created by the Action Proposer alone is not

sufficient to avoid failures. In critical phases with extremely low error tolerance, blind diversity can instead introduce harmful jitters. Because disastrous physical conflicts may be hidden within locally reasonable candidate actions, once the system randomly selects these failure-prone actions, it will lead to irreversible collisions. Therefore, the system must rely on an evaluation mechanism capable of explicitly foreseeing the physical consequences of actions to pick out safe action paths from diverse hypotheses.

Table 2: Ablation study results.

Method	Components		Task Success Rate (%)				Average
	Action Proposer	Dream Evaluator	Cup	Charger	Cap	Screw	
$\pi_{0.5}$	×(Single ODE)	×	62.5	47.5	55.0	30.0	48.8
DREAMAVOID Variants	×(Repeated ODE)	✓	67.5	55.0	70.0	37.5	57.5
	✓(SDE)	×(Random Select)	65.0	35.0	55.0	25.0	45.0
DA-ABL	✓	✓	90.0	67.5	80.0	52.5	72.5

6 LIBERO and SimplerEnv Benchmarks

We evaluate on two simulation benchmarks: LIBERO [16] and SimplerEnv [17]. The base policy adopted for LIBERO is $\pi_{0.5}$, while SimplerEnv uses GR00T-N1.6 [30]. This design aims to verify that DREAMAVOID can generalize to other flow policies. Unlike real-world experiments that rely on human-annotated visual frames, we directly leverage the precise underlying physical states of the simulator to locate the critical phase (Appendix E). Experimental results show (Table 6 and Table 7, Appendix E) that the DA-ABL method achieves an average success rate of 97.8% on LIBERO (outperforming the base policy $\pi_{0.5}$ ’s 96.5% and GPC-RANK’s 96.6%), and attains average performances of 63.6% and 80.7% on SimplerEnv’s Bridge and Fractal datasets, respectively. This comprehensively surpasses both the base policy GR00T-N1.6 (59.9% and 76.4%, respectively) and the GPC-RANK method (61.1% and 78.2%, respectively). This fully demonstrates that DREAMAVOID possesses the same advantages in simulation environments, and that autonomous boundary learning can effectively enhance the model’s robustness when facing local deviations in critical phases.

7 Limitation & Future Works

First, adapting the plug-and-play safety layer DREAMAVOID to other policies (world action models or traditional control policies) is a promising direction. Second, although DREAMAVOID strictly limits dense computations to sparse critical phases, generating high-fidelity future videos and scoring multiple candidate actions still introduces latency. Future work could not only employ various video generation acceleration techniques, but also opt to execute forward outcome simulations within a compact latent space. Third, beyond optimizing decision-making at test-time, future work could leverage the high-fidelity videos generated by the world model as a rich source of training data, utilizing methods like advantage-weighted regression to continuously update the base policy itself. A comprehensive discussion of limitations and future work is in Appendix F.

8 Conclusion

This paper explores the issue of failure avoidance in VLA models. Although existing models possess strong intuitive execution capabilities, they still face limitations in anticipating risks. In contrast, explicit spatio-temporal simulation directly provides the foresight necessary to avoid failures. To this end, we propose the DREAMAVOID framework. Through critical-phase test-time dreaming, DREAMAVOID decouples intuitive execution from foresight by simulating the short-horizon futures of candidate actions during critical phases to select the optimal action. To alleviate the challenge of state evaluation in complex scenarios, we introduce an autonomous boundary learning paradigm, enabling the model to better grasp the boundaries between success and failure. Experiments demonstrate that DREAMAVOID can effectively intercept local errors before they escalate. In summary, we emphasize that test-time dreaming matters; integrating intuitive execution with selective dreaming at test time is an effective pathway to enhancing the reliability of VLAs. Through critical-phase dreaming, we can achieve more robust error avoidance capabilities without a significant increase in inference costs.

References

- [1] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, et al. Openvla: An open-source vision-language-action model. In *Conference on Robot Learning*, pages 2679–2713. PMLR, 2025.
- [2] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, brian ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In Joseph Lim, Shuran Song, and Hae-Won Park, editors, *Proceedings of The 9th Conference on Robot Learning*, volume 305, pages 17–40. PMLR, 2025. URL <https://proceedings.mlr.press/v305/black25a.html>.
- [3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
- [4] Charles Xu, Jost Tobias Springenberg, Michael Equi, Ali Amin, Adnan Esmail, Sergey Levine, and Liyiming Ke. RL token: Bootstrapping online rl with vision-language-action models. *arXiv preprint arXiv:2604.23073*, 2026.
- [5] Heng Zhang, Rui Dai, Gokhan Solak, Pokuang Zhou, Yu She, and Arash Ajoudani. Safe learning for contact-rich robot tasks: A survey from classical learning-based methods to safe foundation models. *arXiv preprint arXiv:2512.11908*, 2025.
- [6] Songqiao Hu, Zeyi Liu, Shuang Liu, Jun Cen, Zihan Meng, and Xiao He. Vlsa: Vision-language-action models with plug-and-play safety constraint layer. *arXiv preprint arXiv:2512.11891*, 2025.
- [7] Yujiro Onishi, Ryo Takizawa, Yoshiyuki Ohmura, and Yasuo Kuniyoshi. Exploration-assisted bottleneck transition toward robust and data-efficient deformable object manipulation. *arXiv preprint arXiv:2603.13756*, 2026.
- [8] Shenyuan Gao, William Liang, Kaiyuan Zheng, Ayaan Malik, Seonghyeon Ye, Sihyun Yu, Wei-Cheng Tseng, Yuzhu Dong, Kaichun Mo, Chen-Hsuan Lin, et al. Dreamdojo: A generalist robot world model from large-scale human videos. *arXiv preprint arXiv:2602.06949*, 2026.
- [9] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. Robomoney: Scaling test-time sampling and verification for vision-language-action models. In *Conference on Robot Learning*, pages 3200–3217. PMLR, 2025.
- [10] Han Qi, Haocheng Yin, Aris Zhu, Yilun Du, and Heng Yang. Inference-time enhancement of generative robot policies via predictive world modeling. *IEEE Robotics and Automation Letters*, 2026.
- [11] Chelsea Finn and Sergey Levine. Deep visual foresight for planning robot motion. In *2017 IEEE international conference on robotics and automation (ICRA)*, pages 2786–2793. IEEE, 2017.
- [12] Wenkai Guo, Guanxing Lu, Haoyuan Deng, Zhenyu Wu, Yansong Tang, and Ziwei Wang. Vla-reasoner: Empowering vision-language-action models with reasoning via online monte carlo tree search. *arXiv preprint arXiv:2509.22643*, 2025.
- [13] Yalcin Tur, Jalal Naghiyev, Haoquan Fang, Wei-Chuan Tsai, Jiafei Duan, Dieter Fox, and Ranjay Krishna. RECURRENT-DEPTH VLA: IMPLICIT TEST-TIME COMPUTE SCALING OF VISION-LANGUAGE-ACTION MODELS VIA LATENT ITERATIVE REASONING. In *Workshop on Latent & Implicit Thinking – Going Beyond CoT Reasoning*, 2026. URL <https://openreview.net/forum?id=hsIm52gD9p>.

- [14] Yanting Yang, Shenyuan Gao, Qingwen Bu, Li Chen, and Dimitris N Metaxas. Seeing farther and smarter: Value-guided multi-path reflection for vlm policy optimization. *arXiv preprint arXiv:2602.19372*, 2026.
- [15] Kun Wu, Ning Liu, Zhen Zhao, Di Qiu, Jinming Li, Zhengping Che, Zhiyuan Xu, and Jian Tang. Learning from imperfect demonstrations with self-supervision for robotic manipulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 16899–16906. IEEE, 2025.
- [16] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [17] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Oier Mees, Karl Pertsch, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, et al. Evaluating real-world robot manipulation policies in simulation. In *Conference on Robot Learning*, pages 3705–3728. PMLR, 2025.
- [18] Changyu Liu, Yiyang Liu, Taowen Wang, Qiao Zhuang, James Chenhao Liang, Wenhao Yang, Renjing Xu, Qifan Wang, Dongfang Liu, and Cheng Han. On-the-fly vla adaptation via test-time reinforcement learning. *arXiv preprint arXiv:2601.06748*, 2026.
- [19] Zechen Bai, Chen Gao, and Mike Zheng Shou. Evolve-vla: Test-time training from environment feedback for vision-language-action models. *arXiv preprint arXiv:2512.14666*, 2025.
- [20] Yunhai Feng, Jiaming Han, Zhuoran Yang, Xiangyu Yue, Sergey Levine, and Jianlan Luo. Reflective planning: Vision-language models for multi-stage long-horizon robotic manipulation. In *Conference on Robot Learning*, pages 2038–2062. PMLR, 2025.
- [21] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [22] The Gemini Team. Gemini 3.1 pro: A smarter model for your most complex tasks. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/>, 2026.
- [23] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [24] Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di ZHANG, and Wanli Ouyang. Flow-grpo: Training flow matching models via online rl. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [25] Kang Chen, Zhihao Liu, Tonghe Zhang, Zhen Guo, Si Xu, Hao Lin, Hongzhi Zang, Quanlu Zhang, Zhaoifei Yu, Guoliang Fan, et al. π rl: Online rl fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889*, 2025.
- [26] Anthony Liang, Yigit Korkmaz, Jiahui Zhang, Minyoung Hwang, Abrar Anwar, Sidhant Kaushik, Aditya Shah, Alex S Huang, Luke Zettlemoyer, Dieter Fox, et al. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *arXiv preprint arXiv:2603.02115*, 2026.
- [27] Peter J Huber. Robust estimation of a location parameter. In *Breakthroughs in statistics: Methodology and distribution*, pages 492–518. Springer, 1992.
- [28] Yanjiang Guo, Tony Lee, Lucy Xiaoyang Shi, Jianyu Chen, Percy Liang, and Chelsea Finn. Vlaw: Iterative co-improvement of vision-language-action policy and world model. *arXiv preprint arXiv:2602.12063*, 2026.
- [29] Tenny Yin, Zhiting Mei, Zhonghe Zheng, Miyu Yamane, David Wang, Jade Sceats, Samuel M Bateman, Lihan Zha, Apurva Badithela, Ola Shorinwa, et al. Playworld: Learning robot world models from autonomous play. *arXiv preprint arXiv:2603.09030*, 2026.

- [30] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [31] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *arXiv preprint arXiv:2410.00371*, 2024.
- [32] Zeyi Liu, Arpit Bahety, and Shuran Song. Reflect: Summarizing robot experiences for failure explanation and correction. In *Conference on Robot Learning*, pages 3468–3484. PMLR, 2023.
- [33] Qiao Gu, Yuanliang Ju, Shengxiang Sun, Igor Gilitschenski, Haruki Nishimura, Masha Itkina, and Florian Shkurti. SAFE: Multitask failure detection for vision-language-action models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=XPyAukgsFf>.
- [34] Huanyu Li, Kun Lei, Sheng Zang, Kaizhe Hu, Yongyuan Liang, Bo An, Xiaoli Li, and Huazhe Xu. Failure-aware rl: Reliable offline-to-online reinforcement learning with self-recovery for real-world manipulation. *arXiv preprint arXiv:2601.07821*, 2026.
- [35] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. In *ICLR 2026 the 2nd Workshop on World Models: Understanding, Modelling and Scaling*, 2026.
- [36] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- [37] Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. *arXiv preprint arXiv:2506.08009*, 2025.

A Extended Related Work

Failure Detection. Handling failures is typically viewed as a reactive process: the system must first detect the occurrence of an error and subsequently attempt to correct it. For instance, AHA [31] introduces an instruction-fine-tuned VLM specifically designed to detect and reason about manipulation failures using natural language. REFLECT [32] explains failures and proposes corrective suggestions by summarizing the robot’s experiences. Furthermore, SAFE proposes a multi-task failure detector for general-purpose VLAs, which predicts the probability of task failure in real-time by analyzing the model’s internal latent features [33]. While these post-hoc reasoning capabilities are invaluable for offline policy improvement, reward generation, and sub-task verification, they inherently operate only after a failure has already occurred.

To achieve a safer execution process, our work shifts the reasoning focus of the system from reactive failure detection to proactive avoidance. Instead of waiting for a failure to happen before reasoning, DREAMAVOID utilizes test-time dreaming to explicitly forward-simulate the consequences of candidate actions. By invoking this failure-aware evaluation at critical phases, DREAMAVOID is able to intercept minor action deviations before they accumulate into irreversible failures. Consequently, our approach acts as a preventive safety layer, effectively complementing the aforementioned works.

World Models. World models provide a natural mechanism for evaluating the downstream consequences of candidate actions. DreamDojo shows that action-conditioned world models, pretrained on large-scale video data and then post-trained on robot data, can support physically meaningful future prediction [8]. Prior work such as VLAW [28] and FARL [34] show that failed trajectories are important for improving robot robustness, recovery ability, and feasibility judgment.

B Pilot Study Details

To evaluate the model’s reasoning capabilities during the critical phase of fine-grained manipulation, we establish a standardized evaluation setup.

Common Setup. Human annotators manually identified a critical interaction frame t_{crit} in each test trajectory. At each t_{crit} , we extract the current frame o_t along with a historical observation sequence consisting of 5 frames with a sampling stride of 8. Each frame is constructed by concatenating three camera views (the main view and the left and right wrist views) into a 2×2 grid (with the bottom-right corner filled in black).

At this moment, the base policy $\pi_{0.5}$ predicts an action chunk of length $H = 50$ (approximately 1.67 s in a 30 fps video). The action consists of 6-dimensional absolute joint angles and the gripper state for both the left and right arms, totaling 14 dimensions. To align with the inference frequency of the world model and reduce the text comprehension burden on the VLM, we downsample the future action chunk using a stride of 4, extracting 13 actions with indices $[0, 4, 8, \dots, 48]$. This sparse action sequence u_t spans 48 time steps, corresponding to an execution window of approximately 1.6 s into the future.

Setup for Explicit Spatiotemporal Dreaming. The historical observation stride is 8, and the future action sequence stride is 4. A sparser visual history is sufficient to capture the macroscopic motion context. However, accurately predicting the physical evolution of the critical phase, which is sensitive to minute deviations, requires denser action-conditioned inputs to ensure that the world model can perform precise spatiotemporal rendering. The world model DreamDojo is fine-tuned on 100 human teleoperation trajectories. Figure 5 illustrates the actual execution alongside the future frames predicted by the world model.

C DREAMAVOID Framework Details

The complete execution procedure of DREAMAVOID is summarized in Algorithm 1.

Algorithm 1 DREAMAVOID

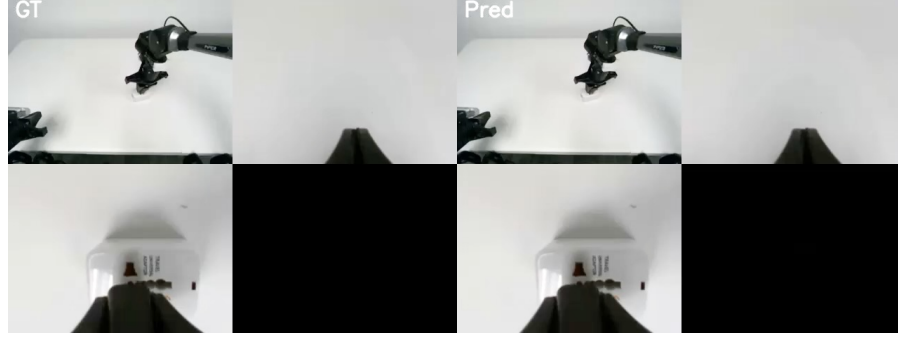
```

1: Observe current state  $o_t$  and instruction  $l$ 
2: Compute criticality score  $p_t = f_\phi(\mathcal{O}_{t-K+1:t}, s_t)$  {Dream Trigger}
3: if  $p_t < \gamma$  then
4:   Execute default action chunk  $u_t \sim \pi_\theta(\cdot \mid o_t, l)$  {Standard Execution}
5: else
6:   Sample diverse candidates  $\{u_t^{(k)}\}_{k=1}^{N_c}$  via SDE solver from  $\pi_\theta(\cdot \mid o_t, l)$  {Action Proposer}
7:   for  $k = 1$  to  $N_c$  do
8:     Dream future consequence  $\hat{y}_t^{(k)} = \mathcal{W}_\psi(o_t, u_t^{(k)})$  {Dream Evaluator}
9:     Evaluate consequence  $\hat{v}_t^{(k)} = V_\omega(o_t, u_t^{(k)}, \hat{y}_t^{(k)})$ 
10:   end for
11:   Select  $k^* = \arg \max_k \hat{v}_t^{(k)}$  {Select the highest value}
12:   Execute  $u_t^{(k^*)}$ 
13: end if

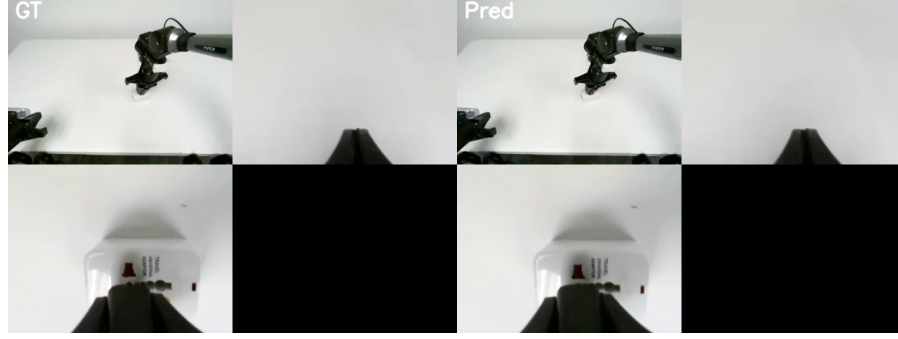
```

C.1 Dream Trigger

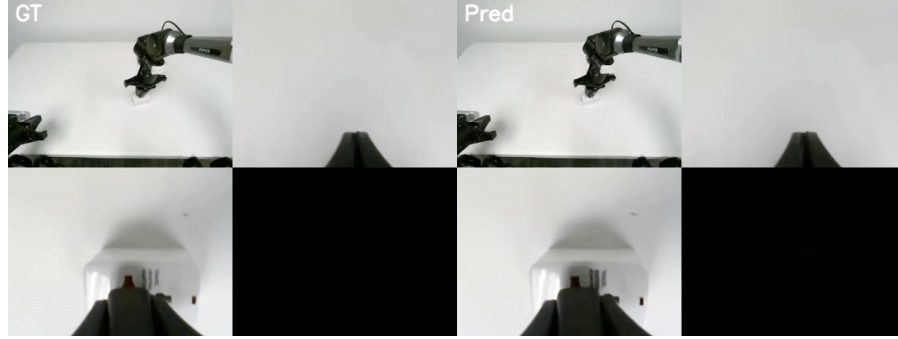
Considering the requirements for real-time inference speed, we select the frozen DINOv2-ViT-B/14 [36] as the backbone for Dream Trigger. First, we perform frame-by-frame feature encoding on the images from each camera feed (which include sparsely sampled historical observations and the current frame) and execute average pooling along the temporal dimension. Subsequently, the aggregated single- or multi-camera visual features are concatenated with the current proprioceptive state s_t and jointly fed into a three-layer MLP classification head to determine the current phase state. In real-world deployment, we configure Dream Trigger to poll asynchronously at a frequency of 1 Hz, which is shorter than the 1.67 s execution cycle of the action chunk. Testing shows that on a single RTX 4090 GPU, the single forward inference latency of the Dream Trigger is approximately 50 ms,



(a)



(b)



(c)

Figure 5: (a) shows the current frame t_{crit} , while (b) and (c) show the actual future outcome frames (GT) and the video frames predicted by DreamDojo (Pred). Each 2x2 image block contains different camera views: the top-left is the main view, the top-right is the left wrist view, and the bottom-left is the right wrist view. This concatenation method follows the approach of DreamZero [35] to accommodate the single-image input requirement of DreamDojo.

accounting for only 3% of the entire control cycle. This lightweight design ensures that the additional state monitoring process has virtually no impact on the robot’s real-time, continuous control.

If a task consists of $M > 1$ key stages (e.g., grasping $\rightarrow$ alignment), we extend the single-head prediction to a multi-head format: $\mathbf{p}_t = [p_t^{(1)}, \dots, p_t^{(M)}] = f_\phi(\cdot)$. Each predicted probability $p_t^{(m)}$ corresponds to an independent $t_{crit}^{(m)}$ and a soft label $\tilde{y}_t^{(m)} = \text{Sigmoid}((t - t_{crit}^{(m)})/\beta)$. The total loss is the sum of the Binary Cross Entropy (BCE) for each stage:

$$\mathcal{L}_{trigger}^{multi} = \sum_{m=1}^M \mathcal{L}_{trigger}^{(m)} \quad (5)$$

The inference trigger condition is modified to $c_t^{\text{test}} = \mathbb{I}[\max_m p_t^{(m)} \geq \gamma]$, meaning test-time dreaming is activated if any stage triggers. This multi-head design shares the same backbone as the single-head version, with only the final layer of the MLP expanded from a 1D output to M dimensions.

C.2 Action Proposer

Table 3 details the noise level σ , the number of generated candidate actions N_c , and the action horizon H used by the DREAMAVOID and GPC-RANK baselines during SDE sampling in real-world and simulation experiments. The σ values in the LIBERO benchmark are referenced from π_{RL} [25]. The noise level σ controls the trade-off between sampling diversity and base policy fidelity. N_c affects the size and diversity of the action candidate pool. The action horizon H determines the length of the generated candidate action chunks, a setting that varies depending on the specific task and the requirements of the base policy. To ensure fairness in the experiments, N_c and H are kept consistent between DREAMAVOID and GPC-RANK.

Table 3: Hyperparameters for Action Proposer in real-world and simulation environments, detailing the SDE noise level σ , action candidate counts N_c , and action horizon H . Both DA-Vanilla and DA-ABL share the same parameters. GPC-RANK utilizes its own generative sampling mechanism rather than SDE sampling, hence the SDE noise level σ is not applicable (N/A).

Environment	Method	Task	σ	N_c	H
Real-World	DREAMAVOID	Cup Slewing	0.1	8	50
		Charger Plugging	0.05		
		Cap Opening	0.08		
		Screw Insertion	0.05		
	GPC-RANK	All Tasks	N/A	8	50
Simulation	DREAMAVOID	LIBERO Object & Goal	0.3	4	10
		LIBERO Spatial & 10	0.5		10
		SimplerEnv	0.3		16
	GPC-RANK	LIBERO	N/A	4	10
		SimplerEnv			16

C.3 Dream Evaluator

C.3.1 World Model

The world model is initialized with DreamDojo-2B [8] and post-trained on target robot data. To support low-latency online intervention, we follow the Self Forcing paradigm [37] to further distill the bidirectional teacher world model into an autoregressive student model. This distillation process consists of two stages: (1) Warmup: under a teacher forcing mechanism, the student model’s predictions are regressed onto the ODE trajectories generated by the teacher. (2) Distillation: utilizing a distribution matching method, the student model is trained autoregressively on its own generated context. This student model replaces the bidirectional attention mechanism with a causal attention mechanism and reduces the number of diffusion sampling steps, thereby achieving efficient online rollout generation while maintaining temporal consistency and action controllability. A single 13-frame video inference takes approximately 2 s on a single RTX 4090 GPU.

C.3.2 Value Model

We regress the continuous progress delta within the execution horizon H :

$$z_t^{(k)} = \eta_{t+H} - \eta_t \quad (6)$$

where $\eta_t \in [0, 1]$ is the frame-level progress score provided by Robometer [26]. This continuous label intuitively reflects the true physical contribution of the action $u_t^{(k)}$ towards task completion. To ensure the network can unambiguously identify catastrophic consequences, we introduce a boundary penalty mechanism. If Robometer predicts an irreversible failure within the horizon (e.g., a dropping object or severe collision causing a precipitous drop in success probability μ_t), we forcefully truncate

and reset its target label $z_t^{(k)}$ to -1 . The final layer of the value model directly outputs the predicted progress delta $\hat{v}_t^{(k)}$ via a linear layer. Since the target label $z_t^{(k)}$ is strictly bounded within the $[-1, 1]$ range, the network implicitly learns to constrain its unnormalized outputs to a similar range during training.

D Real-World Evaluation Details

D.1 Real-World Deployment Setup

We deploy our base policy $\pi_{0.5}$ on a single NVIDIA RTX 4090 GPU. Table 4 details the task specifications, training steps (on 8 NVIDIA A100 GPUs), and demonstration data statistics (episodes and average lengths) for the real-world post-training of $\pi_{0.5}$.

Table 4: Detailed task information for the real-world evaluation tasks. All videos in the teleoperation dataset are recorded at 30 FPS.

Task Name	Task Instruction	$\pi_{0.5}$ Training Steps	Teleop Episodes	Avg. Frames
Cup Slewing	Pick up the paper cup and put it into the cup sleeve.	40000	100	330.8
Charger Plugging	Plug the charger into the socket.	50000	100	336.5
Cap Opening	Open the cap of the spray.	50000	100	263.6
Screw Insertion	Insert the screw into the hole in the box.	50000	130	313.6

D.2 Multi-View Input Observations for Base Policy

Figure 6 shows three-view input observations for base policy $\pi_{0.5}$ in real-world tasks.

D.3 Latency Analysis and Smooth Transition

Table 5 details the inference latency of different methods. A single forward pass of the base policy $\pi_{0.5}$ to generate an action chunk of $H=50$ steps takes approximately 93 ms. During the normal execution phase, the Dream Trigger of DREAMAVOID polls the shared GPU at a frequency of 1 Hz (50 ms per poll), executing in parallel with the base policy inference on a different process. Therefore, it does not introduce observable control loop latency (average GPU utilization $<5\%$). Only when entering a critical phase, which typically occurs *only 1 to 3 times per entire trajectory*, does the system synchronously and serially execute the Action Proposer and Dream Evaluator, resulting in a *single intervention* latency of approximately 2.1 s.

In contrast, always-on test-time methods like GPC-RANK require replanning every 25 steps. This means that for a single action chunk (50 steps) of the base policy, it incurs inference latency twice. For trajectories that typically range from 260 to 340 frames in length (Table 4), GPC-RANK requires 11-14 full replanning inferences (*each with a latency of 694 ms*). Because DREAMAVOID strictly restricts dense computation to sparse critical phases (typically triggered only 1-3 times globally), its time cost is heavily amortized, thereby maintaining high execution efficiency overall.

During test-time dreaming, the robot does not trigger an emergency stop, but instead enters a position-holding state. The high-level ROS nodes are blocked, yet the low-level servos continue to actively track the last published setpoint via PID control, naturally damping the velocity to zero. In the contact phase, this mechanism maintains the instantaneous contact force and avoids the mechanical shock of an E-stop. However, for force-sensitive tasks, this is equivalent to applying a sustained, constant contact force. Because the low-level control loop remains active, the PID controller only generates a steady-state error rather than unbounded integral windup. If the robotic arm experiences minor physical drift during the pause due to gravity or payload, our framework re-initializes the action sequence based on the latest proprioceptive state after inference concludes, thereby eliminating command deviation. To mitigate system jitter at the instant execution resumes, we addressed two core issues: First, to handle the spatial jump between the last setpoint and the first frame of the new action, we implemented trajectory blending, ensuring C^0 continuity through forward linear interpolation. Second, the sleep deficits accumulated by the high-level timer can cause an instantaneous burst of commands upon recovery (triggering a velocity step). To resolve this, we explicitly reset the



Figure 6: Multi-view input observations for base policy $\pi_{0.5}$ in real-world tasks (main view | left wrist view | right wrist view). Each row presents three synchronized camera views for a specific task, highlighting the need for accurate spatial alignment and contact establishment. These tasks are sensitive to minor action deviations compared to coarse-grained pick-and-place scenarios.

ROS clock and introduce a stabilization buffer right before recovery, effectively achieving a smooth transition.

E Simulation Benchmark Details

Unlike real-world experiments that rely on human-annotated visual frames, we directly leverage the precise underlying physical states of the simulator to locate the critical phase. Specifically, we convert the triggering condition of the Dream Trigger into a spatial distance metric: when the underlying coordinates indicate that the Euclidean distance between the robot’s end-effector and the target object is less than a set threshold, the system automatically determines that the critical phase has been entered.

Detailed evaluation results on the LIBERO and SimplerEnv benchmarks are shown in Table 6 and Table 7. On SimplerEnv, the different future videos generated by DreamDojo based on 4 candidate action chunks are illustrated in Figure 7.

Table 5: Time consumption analysis in the real-world evaluation tasks.

Method	Component	Hardware Setup	Latency (ms)
Base Policy	$\pi_{0.5}$ Action Chunk Gen. ($H=50$)	1 $\times$ RTX 4090	93 $\pm$ 5
	Est. Total Trajectory Inference (<i>replans every 50 steps, 6–7 times / trajectory</i>)		$\sim 0.6\text{--}0.7$ s
DREAMAVOID	Dream Trigger (~ 1 Hz polling) [†]	1 $\times$ RTX 4090	50 $\pm$ 5
	Action Proposer (SDE Sampling, $K=8$)	(Shared Local)	93 $\pm$ 5
	Dream Evaluator (DreamDojo Video Gen)	3 $\times$ RTX 4090 (Dist.)*	2000 $\pm$ 100
	Dream Evaluator (Value Model Scoring)		40 $\pm$ 5
	Total Critical-Phase Intervention (<i>triggered 1–3 times / trajectory</i>)		2133 $\pm$ 110
	Est. Total Trajectory Inference (<i>Base Policy + Interventions</i>)		$\sim 2.7\text{--}7.0$ s
GPC-RANK	$\pi_{0.5}$ Candidate Sampling ($K=8$)	1 $\times$ RTX 4090	93 $\pm$ 5
	World Model Rollout (8 steps $\times$ 3 denoise, batch 8)		600 $\pm$ 4
	Reward Predictor (ResNet-18, batch 8)		1 $\pm$ 0.6
	Total Per-Replan (<i>replans every 25 steps, 11–14 times / trajectory</i>)		694 $\pm$ 9.6
	Est. Total Trajectory Inference (<i>Base Policy + Interventions</i>)		$\sim 7.6\text{--}9.7$ s

[†] Runs in parallel with $\pi_{0.5}$ on a separate process; not additive to per-step latency.

* 8 parallel instances are co-located on 3 GPUs (~ 7.5 GB VRAM each instance).

Table 6: Comparisons on the LIBERO benchmark. Each task is tested 200 times.

Method	Success Rate (%)				
	Object	Goal	10	Spatial	Average
Maximum Step	280	300	520	220	330
$\pi_{0.5}$	98.0	98.0	91.0	99.0	96.5
GPC-RANK	99.0	97.5	92.0	98.0	96.6
DA-Vanilla	98.5	97.0	93.0	98.5	96.8
DA-ABL	98.5	99.0	94.5	99.0	97.8

Table 7: Comparisons on the SimplerEnv benchmark. Each task is tested 200 times.

Task	GR00T-N1.6	GPC-RANK	DA-Vanilla	DA-ABL
Bridge Dataset (WidowX Robot)		Success Rate (%)		
widowx_spoon_on_towel	62.5	64.5	65.0	67.0
widowx_carrot_on_plate	63.0	64.0	64.0	65.5
widowx_put_eggplant_in_basket	89.5	91.0	92.5	95.0
widowx_stack_cube	4.0	6.0	5.5	7.0
widowx_put_eggplant_in_sink	38.0	38.0	40.5	41.0
widowx_close_drawer	68.0	70.5	70.0	73.5
widowx_open_drawer	94.0	94.0	95.5	96.0
Average	59.9	61.1	61.9	63.6
Fractal Dataset (Google Robot)		Success Rate (%)		
google_robot_pick_coke_can	97.0	97.5	97.0	98.5
google_robot_pick_object	85.0	87.0	88.0	89.5
google_robot_move_near	73.5	74.0	76.0	78.5
google_robot_open_drawer	44.0	45.0	48.5	51.0
google_robot_close_drawer	82.5	87.5	85.0	86.0
Average	76.4	78.2	78.9	80.7

F Limitation & Future Works

First, although DREAMAVOID strictly limits dense computations to sparse critical phases, generating high-fidelity future videos and scoring multiple candidate actions still introduces latency. For high-precision but slower-paced manipulation tasks, this overhead is acceptable. However, in highly dynamic environments requiring split-second reactive control, this latency may hinder the robot’s



Figure 7: On the `google_robot_pick_coke_can` task of the `SimplerEnv` benchmark, future videos are generated by `DreamDojo` based on 4 action chunk candidates produced by `GR00T-N1.6` via `SDE` sampling (13 frames in total; frames 0, 3, 6, 9, and 12 are shown here). Among these, (4) has the highest corresponding value score.

smooth execution. In addition to employing various video generation acceleration techniques, future research could also perform forward consequence simulation within a compact latent space. It must be emphasized, however, that explicitly generating pixel-level videos holds irreplaceable value: its intuitive interpretability not only enables human researchers to quickly diagnose and optimize the model, but these synthesized failure scenarios and corrective trajectories can also directly serve as high-quality negative samples or preference data for the continuous fine-tuning of subsequent VLA or world models (e.g., via DPO or RLHF).

Secondly, although this paper primarily uses `DREAMAVOID` as an enhancement module for VLA models, its core test-time dreaming mechanism is architecturally completely independent of the underlying base policy. Future work could integrate this failure-aware framework as a plug-and-play test-time safety layer into broader embodied control architectures, such as the emerging World Action Models or more traditional control policies. For specific implementations, only adaptive modifications to the action sampling methods of different architectures are required: for instance, `SDE` sampling can be adopted for action trajectory search in flow matching-based policies, while diverse candidate actions can be generated by injecting Gaussian perturbations in diffusion-based models.

G Broader Impacts

Our proposed `DREAMAVOID` framework enhances the safety and robustness of embodied AI systems. By proactively predicting and avoiding failures at critical phases, this method reduces the risk of physical collisions and damage to robotic arms and objects, paving the way for safe interactions in domestic and industrial environments. However, there are also potential negative impacts to consider. The improved reliability and autonomy of robotic systems may accelerate the displacement of human labor in certain manual tasks. Furthermore, robots might execute unexpected or unsafe actions in out-of-distribution scenarios. Therefore, appropriate human supervision remains crucial.